

| 操作系统作业3

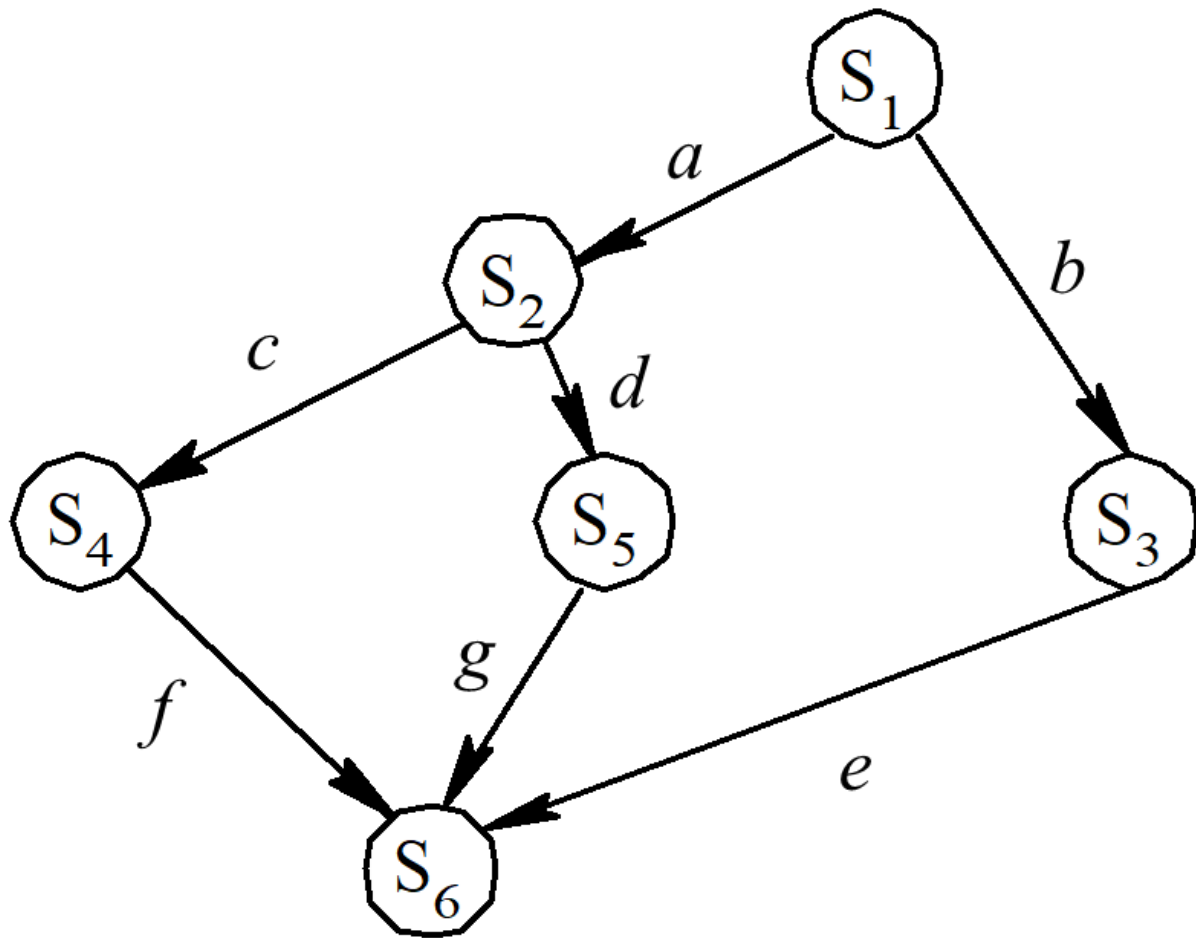
定义信号量PV操作：

```
#include <mutex>
#include <condition_variable>
class Semaphore{
private:
    int count;
    mutex mtx;
    condition_variable cv;
public:
    Semaphore(int initial = 0) : count(initial) {}
    void release(){// V 操作
        unique_lock<mutex> lock(mtx);
        count++;
        cv.notify_one(); // 唤醒一个等待线程
    }

    void acquire() { // P 操作
        unique_lock<mutex> lock(mtx);
        while (count <= 0) {
            cv.wait(lock); // 阻塞直到 count > 0
        }
        count--;
    }
};
```

由于C++20提供了counting_semaphore作为信号量，后续直接使用。

| 具备前驱-后继关系的进程的控制



```
#include <iostream>
#include <thread>
#include <semaphore>
#include <chrono>

using namespace std;

// 定义信号量，初始值都为0
counting_semaphore a(0), b(0), c(0), d(0), e(0), f(0), g(0);

void S1() {
    cout << "Executing S1" << endl;
    this_thread::sleep_for(chrono::milliseconds(100)); // 模拟执行时间
    a.release(); // V(a)
    b.release(); // V(b)
}

void S2() {
    a.acquire(); // P(a)
    cout << "Executing S2" << endl;
    this_thread::sleep_for(chrono::milliseconds(100));
```

```

    c.release(); // V(c)
    d.release(); // V(d)
}

void S3() {
    b.acquire(); // P(b)
    cout << "Executing S3" << endl;
    this_thread::sleep_for(chrono::milliseconds(100));
    e.release(); // V(e)
}

void S4() {
    c.acquire(); // P(c)
    cout << "Executing S4" << endl;
    this_thread::sleep_for(chrono::milliseconds(100));
    f.release(); // V(f)
}

void S5() {
    d.acquire(); // P(d)
    cout << "Executing S5" << endl;
    this_thread::sleep_for(chrono::milliseconds(100));
    g.release(); // V(g)
}

void S6() {
    e.acquire(); // P(e)
    f.acquire(); // P(f)
    g.acquire(); // P(g)
    cout << "Executing S6" << endl;
    this_thread::sleep_for(chrono::milliseconds(100));
}

int main() {
    thread t1(S1), t2(S2), t3(S3), t4(S4), t5(S5), t6(S6);

    t1.join();
    t2.join();
    t3.join();
    t4.join();
    t5.join();
    t6.join();

    cout << "All tasks completed!" << endl;
    return 0;
}

```

```
Executing S1
Executing S2
Executing S3
Executing S4
Executing S5
Executing S6
All tasks completed!
```

父亲、儿子、女儿的同步

桌上有一空盘，最多允许存放一只水果。爸爸可向盘中放一个苹果或放一个桔子，儿子专等吃盘中的桔子，女儿专等吃苹果。试用P、V操作实现爸爸、儿子、女儿三个并发进程的同步
先写一个伪代码：

```
// 信号量初始化
semaphore mutex = 1;           // 互斥访问盘子
semaphore plate_empty = 1;     // 盘子是否为空
semaphore apple_available = 0; // 苹果可用
semaphore orange_available = 0; // 桔子可用

// 爸爸进程
process_dad() {
    while (true) {
        P(plate_empty);        // 等待盘子为空
        P(mutex);              // 获取盘子访问权

        // 随机选择放苹果或桔子
        if (rand() % 2 == 0) {
            put_apple();        // 放苹果
            V(apple_available); // 通知女儿
        } else {
            put_orange();       // 放桔子
            V(orange_available); // 通知儿子
        }

        V(mutex);              // 释放盘子访问权
    }
}
```

```

}

// 儿子进程
process_son() {
    while (true) {
        P(orange_available); // 等待桔子
        P(mutex);           // 获取盘子访问权

        eat_orange();        // 吃桔子
        V(plate_empty);      // 通知盘子为空

        V(mutex);           // 释放盘子访问权
    }
}

// 女儿进程
process_daughter() {
    while (true) {
        P(apple_available); // 等待苹果
        P(mutex);           // 获取盘子访问权

        eat_apple();         // 吃苹果
        V(plate_empty);      // 通知盘子为空

        V(mutex);           // 释放盘子访问权
    }
}

```

```

#include <iostream>
#include <thread>
#include <semaphore>
#include <chrono>

using namespace std;

// 定义信号量，初始值都为0
counting_semaphore plate_empty(1), use_plate(1), apple(0), orange(0);

void father(){
    while(1) {
        plate_empty.acquire();//P
    }
}

```



```

        use_plate.acquire();//用盘子
        if (rand() % 2 == 0) {
            cout << "父亲放了苹果" << endl;
            apple.release();
        } else {
            cout << "父亲放了橘子" << endl;
            orange.release();
        }
        use_plate.release();
        this_thread::sleep_for(chrono::seconds(1));
    }
}

void son(){
    while(1) {
        orange.acquire();
        use_plate.acquire();

        cout << "儿子拿走了橘子" << endl;
        plate_empty.release();
        use_plate.release();
        this_thread::sleep_for(chrono::seconds(1));
    }
}

void daughter(){
    while(1) {
        apple.acquire();
        use_plate.acquire();

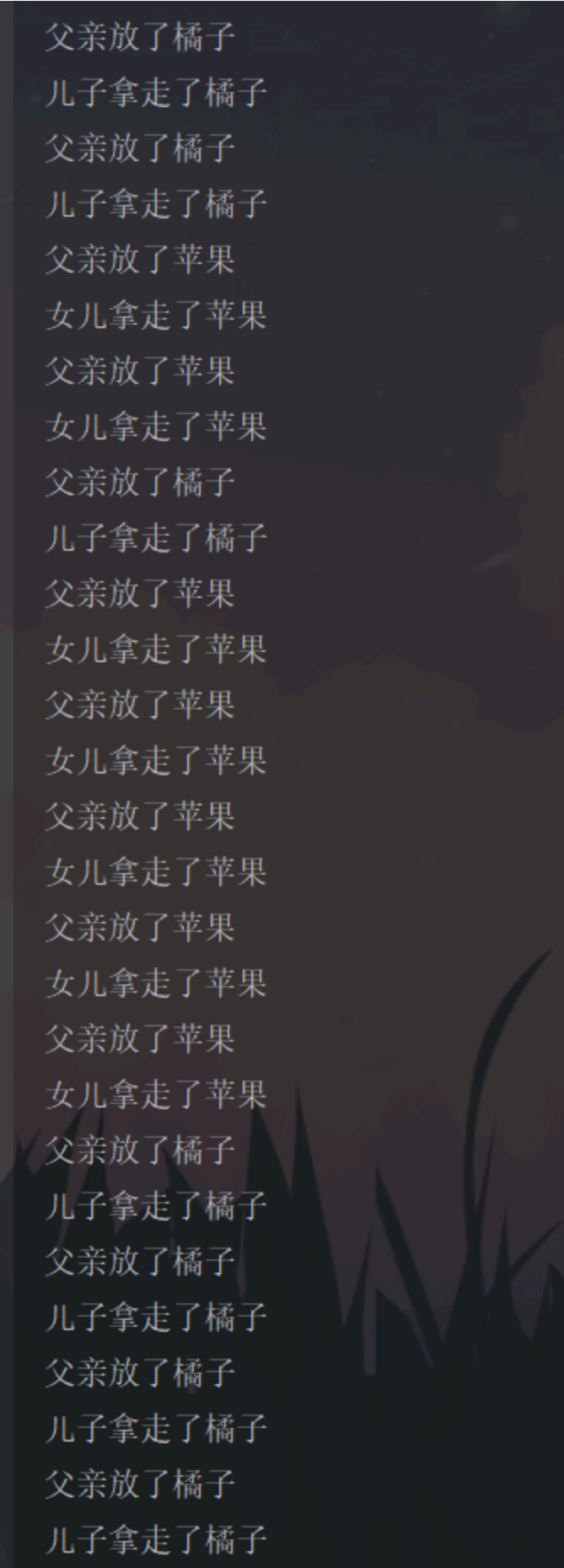
        cout << "女儿拿走了苹果" << endl;
        plate_empty.release();
        use_plate.release();
        this_thread::sleep_for(chrono::seconds(1));
    }
}

int main() {
    srand(time(NULL)); // 设置随机种子（以当前时间）
    thread t1(father), t2(son), t3(daughter);

    t1.join();
    t2.join();
    t3.join();
    cout << "All tasks completed!" << endl;
}

```

```
    return 0;  
}
```



父亲放了橘子
儿子拿走了橘子
父亲放了橘子
儿子拿走了橘子
父亲放了苹果
女儿拿走了苹果
父亲放了苹果
女儿拿走了苹果
父亲放了橘子
儿子拿走了橘子
父亲放了苹果
女儿拿走了苹果
父亲放了苹果
女儿拿走了苹果
父亲放了苹果
女儿拿走了苹果
父亲放了苹果
女儿拿走了苹果
父亲放了橘子
儿子拿走了橘子
父亲放了橘子
儿子拿走了橘子
父亲放了橘子
儿子拿走了橘子
父亲放了橘子
儿子拿走了橘子

拥挤路段的控制

有左行的车，有右行的车，左行占了之后就只能左不能右，右行同理。只有下面一条路。



伪代码：

```
// 全局变量
mutex = Semaphore(1)           // 互斥锁
left_lock = Semaphore(1)       // 左行锁
right_lock = Semaphore(1)      // 右行锁
left_count = 0                 // 左行车辆数
right_count = 0                // 右行车辆数

// 左行车辆
process_left_car() {
    P(left_lock)                // 尝试获取左行锁
    P(mutex)                    // 进入临界区
    left_count++
    if (left_count == 1) {
        P(right_lock)          // 如果是第一辆左行车，禁止右行
    }
    V(mutex)                   // 离开临界区
    V(left_lock)                // 允许其他左行车进入

    // 左行通过...
    print("左行通过")

    P(mutex)                   // 进入临界区
```

```

    left_count--
    if (left_count == 0) {
        V(right_lock)        // 如果没有左行车，允许右行
    }
    V(mutex)                 // 离开临界区
}

// 右行车辆
process_right_car() {
    P(right_lock)            // 尝试获取右行锁
    P(mutex)
    right_count++
    if (right_count == 1) {
        P(left_lock)        // 如果是第一辆右行车，禁止左行
    }
    V(mutex)
    V(right_lock)

    // 右行通过...
    print("右行通过")

    P(mutex)
    right_count--
    if (right_count == 0) {
        V(left_lock)        // 如果没有右行车，允许左行
    }
    V(mutex)
}

```

```

#include <iostream>
#include <thread>
#include <semaphore>
#include <chrono>

using namespace std;

// 定义信号量，初始值都为0
counting_semaphore road(1), leftlock(1), rightlock(1);
int lcount=0, rcount=0;

void left_car(int id){

```

```

    leftlock.acquire();
    road.acquire();
    lcount++;
    if (lcount == 1) {
        rightlock.acquire();
    }
    cout << "左行"<<id<<"号车通过" << endl;

    road.release();
    leftlock.release();
    this_thread::sleep_for(chrono::seconds(1));
    road.acquire();
    lcount--;
    if (lcount == 0) {
        rightlock.release();
    }
    road.release();
}

```

```

void right_car(int id){

    rightlock.acquire();
    road.acquire();
    rcount++;
    if (rcount == 1) {
        leftlock.acquire();
    }
    cout << "右行"<<id<<"号车通过" << endl;

    road.release();
    rightlock.release();
    this_thread::sleep_for(chrono::seconds(1));
    road.acquire();
    rcount--;
    if (rcount == 0) {
        leftlock.release();
    }
    road.release();
}

```

```

int main() {
    srand(time(NULL)); // 设置随机种子（以当前时间）
    thread t1(left_car, 1), t2(left_car, 2), t3(left_car, 3), t4(left_car, 4);
}

```

```
    thread t5(right_car, 1), t6(right_car, 2), t7(right_car, 3), t8(right_car,
4);
    t1.join();
    t2.join();
    t5.join();
    t6.join();
    t3.join();
    t4.join();
    t7.join();
    t8.join();
    cout << "All tasks completed!" << endl;
    return 0;
}
```

左行1号车通过

左行2号车通过

左行3号车通过

左行4号车通过

右行3号车通过

右行2号车通过

右行1号车通过

右行4号车通过

All tasks completed!